



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of
Computing

SECP3623

Database Programming 2025/2026

SEMESTER 1

Assignment 2 (10%) – Transaction Management

Name	:	IMAN ABADI BIN MOHD NIZWAN
Matric Id	:	A23CS0084
Section	:	02

Name	:	MOHAMED ALIF FATHI BIN ABDUL LATIF
Matric Id	:	A23CS0112
Section	:	02

Instructions:

This assignment evaluates your ability to apply the concept of serializability, differentiate between serial and non-serial transaction schedules and implement locking techniques to manage simultaneous transactions.

Academic Integrity

All students must work in a group of **TWO(2)**. The use of AI-generated tools or sharing of answers between students **is strictly prohibited** and will result in **zero marks**. You are required to complete the answers in **handwritten form**. Please take **clear screenshots** of your answers and submit them in **PDF format**.

QUESTION 1 (15 MARKS)

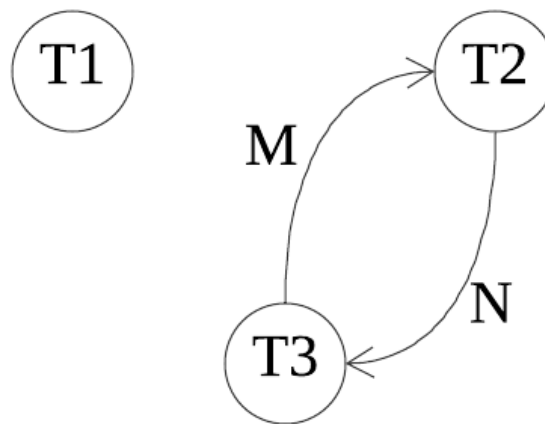
Consider the schedule transactions in Table 1 and answer all following questions (i) – (iv).

Table 1: Schedule T1, T2 and T3

Time	T1	T2	T3
t ₁	begin transaction		
t ₂	read_item(X)		
t ₃	read_item(Y)	begin transaction	
t ₄	commit	read_item(M)	begin transaction
t ₅		read_item(N)	read_item(N)
t ₆		$M = (N + M) * 10$	read_item(M)
t ₇		write_item(M)	$N = N + 150$
t ₈		commit	write_item(N)
t ₉			commit

- i) Draw the precedence graph for the transaction schedule in Table 1 to check for serializability.

Answer:



- ii) Explain the precedence graph produced in (i).

Answer:

Edge (T3, T2) shows T3 reads item M at t_6 before T2 writes item M at t_7 , which is a conflict. **Edge (T2, T3)** shows T2 reads item N at t_5 before T3 writes item N at t_8 , which is also a conflict. There is a cycle in the graph, so it is not conflict serializable.

- iii) Apply the locking technique to the transaction schedule in Table 1, and revise the transaction schedule in Table 1 using the following format:

T1	T2	T3

Ans:

Time	T1	T2	T3
<i>t1</i>	begin transaction		
<i>t2</i>	s-lock(X)		
<i>t3</i>	read(Y)	begin transaction	
<i>t4</i>	unlock(X)	x-lock(M)	begin transaction
<i>t5</i>	s-lock(Y)	read(M)	x-lock(N)
<i>t6</i>	read(Y)	s-lock(N)	read(N)
<i>t7</i>	commit/unlock(Y)	WAIT	s-lock(M)
<i>t8</i>		WAIT	WAIT
<i>t9</i>		WAIT	WAIT
<i>t10</i>		WAIT	WAIT

- iv) Identify the problem(s) that arise in (iii) and then explain any potential problems that may still occur after applying the locking technique in concurrency control based on your answer in (iii).

Answer:

The problem that arises in (iii) is deadlock. At *t7*, T2 tries to write to item M. It needs an exclusive lock on item M, but it must wait because T3 holds a shared lock on item M at *t6*, making T2 wait for T3. At *t8*, T3 tries to write to item N. It needs an exclusive lock on item N, but it must wait because T2 currently holds a shared lock on item N at *t5*, making T3 wait for T2 as well. The circular wait results in a deadlock where neither of the transactions can access the item they want to complete their transaction. There is also a potential for starvation and a livelock problem to happen after applying the locking technique in concurrency control.

QUESTION 2 (15 MARKS)

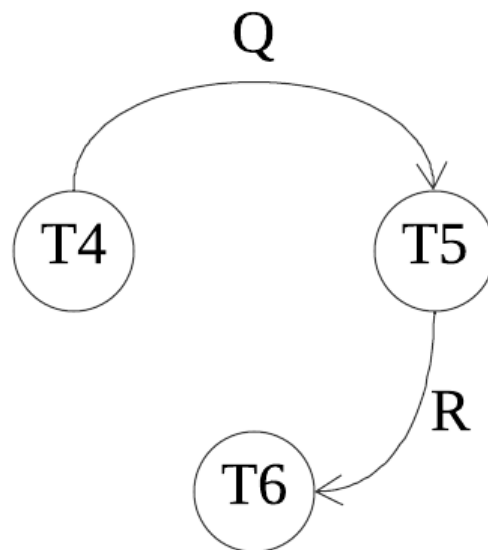
Based on the transaction schedule in Table 2, answer all the following questions:

Table 2: Transaction Schedule for T4-T5-T6

	T4	T5	T6
↓ time	begin transaction		
	read_item(P)		
	read_item(Q)	begin transaction	
	$P=(P*5)+Q$	read_item(Q)	
	write_item(P)	read_item(R)	begin transaction
	commit	$Q=Q+R$	read_item(R)
		write_item(Q)	$R=R-100$
		commit	write_item(R)
			commit

- i) Draw the precedence graph for the transaction schedule in Table 2 to check for serializability.

Answer:



- ii) Explain the precedence graph produced in (i).

Answer:

Edge (T4,T5) shows a conflict on item Q because T4 reads Q at t_3 before T5 writes item Q at t_7 . To maintain consistency, T4 must finish with item Q before T5 changes it. This establishes a dependency where T4 must be done first before T5 can proceed. Since the graph contains no cycles, the schedule is conflict serializable.

- iii) Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

T4	T5	T6

Answer:

Time	T1	T2	T3
<i>t1</i>	begin transaction		
<i>t2</i>	x-lock(P)		
<i>t3</i>	read(P)	begin transaction	
<i>t4</i>	s-lock(P)	x-lock(Q)	
<i>t5</i>	read(Q)	WAIT	begin transaction
<i>t6</i>	$P = (P*5) + Q$	WAIT	x-lock(R)
<i>t7</i>	write(P)	WAIT	read(R)
<i>t8</i>	commit/unlock(P,Q)	WAIT	$R = R - 100$
<i>t9</i>		read(Q)	write (R)
<i>t10</i>		s-lock(R)	commit/unlock(R)
<i>t11</i>		read(R)	
<i>t12</i>		$Q = Q+R$	
<i>t13</i>		write(Q)	
<i>t14</i>		commit/unlock(Q, R)	

QUESTION 3 (25 MARKS)

a) Consider the schedule transactions in Table 3.1 and answer all following questions (i) – (iv).

Table 3.1: Schedule T7, T8, T9 and T10

Time	T7	T8	T9	T10
t ₁	begin transaction	begin transaction		
t ₂	read (X)	read (Y)	begin transaction	
t ₃	$X = X + 50$		read (X)	begin transaction
t ₄	write (X)			read (Z)
t ₅		read (X)	$X = X - 15$	$Z = Z + 20$
t ₆		$X = Y * 12$	write (X)	write (Z)
t ₇		write (X)	commit	
t ₈	commit	commit		commit

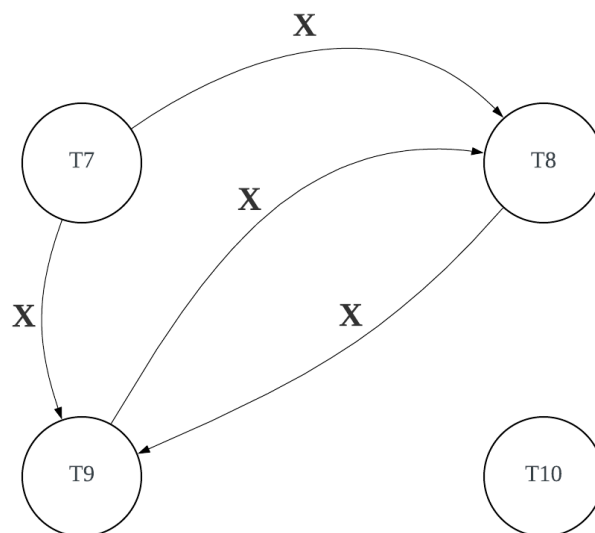
i) Is the schedule in Table 3.1 a serial or non-serial schedule? State your reason.

Answer:

Non-serial schedule because the operations of T7, T8, T9 and T10 overlap in time. A serial schedule executes one transaction completely before another starts while this schedule does not.

ii) Draw the precedence graph for the transactions given in Table 3.1.

Answer:



- iii) Does the schedule in Table 3.1 have conflict-serializable or not? How can you determine this?

Answer:

Not conflict-serializable because the graph contain a cycle

- iv) What problem does the schedule in Table 3.1 face?

Answer:

Lost Update Problem

- b) Consider the schedule transactions in Table 3.2 and answer all following questions (i) – (iv).

Table 3.2: Schedule T11, T12, T13 and T14

Time	T11	T12	T13	T14
t ₁	begin transaction			
t ₂	read (A)	begin transaction		
t ₃		read (B)	begin transaction	
t ₄	A = A + 200	B = B - 50	read (A)	begin transaction
t ₅	write (A)			read (C)
t ₆		write (B)	A = A - 30	C = C + 10
t ₇			write (A)	write (C)
t ₈	commit			
t ₉		read (A)		
t ₁₀		A = A * 2		
t ₁₁		write (A)		
t ₁₂		commit	commit	commit

- i) Implement the basic locking to the schedule in Table 3.2 by specifying the appropriate lock type (shared or exclusive) for each transaction.

Answer:

Transaction	Data Item	Lock Type
T11	A	Exclusive lock
T12	B	Exclusive lock
T12	A	Exclusive lock
T13	A	Exclusive lock
T14	C	Exclusive lock

- ii) Based on your answer in (i), state any other issues that may arise.

Answer:

Deadlock

- iii) Implement the Two-Phase Locking (2PL) protocol to the schedule in Table 3.2.

Answer:

Time	T11	T12	T13	T14
t₁	begin transaction			
t₂	write_lock(A)	begin transaction		
t₃	read(A)	write_lock(B)	begin transaction	
t₄		read(B)	write_lock(A)	begin transaction
t₅	A = A + 200	B = B - 50	WAIT	write_lock(C)
t₆	write(A)		WAIT	read(C)
t₇		write(B)	WAIT	C = C + 10
t₈			WAIT	
t₉	commit / unlock (A)		WAIT	
t₁₀		write_lock(A)	read(A)	
t₁₁		WAIT		
t₁₂		WAIT	A = A - 30	commit / unlock (C)
t₁₃		WAIT	write(A)	
t₁₄		WAIT		
t₁₅		WAIT		
t₁₆		WAIT		
t₁₇		WAIT	commit / unlock (A)	
t₁₈		read(A)		
t₁₉		A = A * 2		
t₂₀		write(A)		
t₂₁		commit / unlock (A,B)		

- iv) Based on your answer in (iii), state any other issues that may arise.

Answer:

Cascading Rollback

QUESTION 4 (15 MARKS)

Consider the schedule transactions in Table 4 and answer all following questions (i) – (v).

Table 4.1: Schedule T15, T16 and T17

Time	T15	T16	T17
t ₁	begin transaction		
t ₂	read (J)	begin transaction	
t ₃	J = J + 10	read (K)	begin transaction
t ₄	write (J)	K = K - 5	read (J)
t ₅	Read (K)	write (K)	J = J - 15
t ₆	K = K + 20		write (J)
t ₇	write (K)	commit	
t ₈	commit		commit

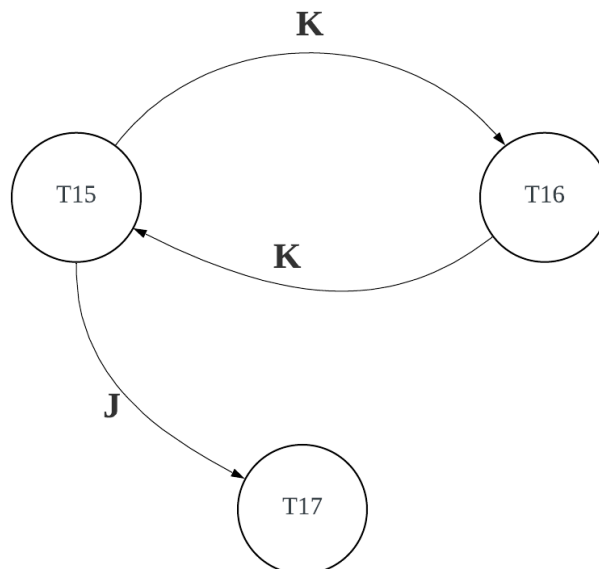
- i) Is the schedule in Table 4 a serial or non-serial schedule? State your reason.

Answer:

Non-serial schedule because the operations of T15, T16 and T17 overlap in time. A serial schedule executes one transaction completely before another starts while this schedule does not.

- ii) Draw the precedence graph for the transactions given in Table 4.

Answer:



- iii) Does the schedule in Table 4 have conflict-serializable or not? How can you determine this?

Answer:

Not conflict-serializable because the graph contain a cycle

- iv) What problem does the schedule in Table 4 face?

Answer:

Lost Update Problem

- v) Apply the two-phase locking (2PL) protocol to emphasize the positioning of lock and unlock operations in each transaction using the following format.

Answer:

Time	T15	T16	T17
t ₁	begin transaction		
t ₂	write_lock(J)	begin transaction	
t ₃	read(J)	write_lock(K)	begin transaction
t ₄	J = J + 10	read(K)	write_lock(J)
t ₅	write(J)	K = K - 5	WAIT
t ₆	write_lock(K)	write(K)	WAIT
t ₇	WAIT		WAIT
t ₈	WAIT	commit / unlock (K)	WAIT
t ₉	read(K)		WAIT
t ₁₀	K = K + 20		WAIT
t ₁₁	write(K)		WAIT
t ₁₂	commit / unlock (J,K)		WAIT
t ₁₃			read(J)
t ₁₄			J = J - 15
t ₁₅			write(J)
t ₁₆			
t ₁₇			commit / unlock (J)